

DevLab Interface Library

Documentación Técnica

23 de septiembre de 2026 · Desarrollador: Jonathan Mejorado

UNIT Electronics — DevLab Ecosystem

https://github.com/UNIT-Electronics-MX/unit_devlab_interface_library

Resumen

DevLab Interface es la capa de acceso a buses del ecosistema DevLab de UNIT Electronics: una biblioteca de Arduino/C++ que define un único contrato de acceso a registros (lectura/escritura de 8, 16 y 32 bits, más helpers de bit-field) y permite que cualquier transporte físico — I2C hoy, SPI en desarrollo — lo implemente con solo dos primitivas crudas (`writeBytes`, `readBytes`).

La idea central: **un driver de sensor se escribe una sola vez contra el contrato, y corre sin cambios sobre cualquier bus que lo implemente**. Hoy eso cubre I2C de 7 y 10 bits, con recuperación automática de bus ante fallos, y soporte probado en ESP32, ESP32-C6/C5/H2, RP2040 y STM32. SPI está planeado como el siguiente transporte concreto.

En versión **1.1.2**, la biblioteca ya es la base de referencia para integrar sensores externos y dispositivos propios de DevLab (como el protocolo DDP de direccionamiento dinámico) sin acoplar cada driver a un bus o a una plataforma específica. Este documento explica la arquitectura, el estado actual y hacia dónde va — no es un manual función por función; esa referencia vive en el código y en los ejemplos del repositorio.

El problema que resuelve

El ecosistema DevLab integra sensores propios y de terceros (Bosch BMI323, MPU6050, etc.) sobre múltiples microcontroladores (ESP32, RP2040, STM32). Sin una capa común, cada combinación sensor $\times$ bus $\times$ plataforma tiende a producir su propio código de acceso a registros: distintas formas de escribir un puntero de registro, hacer un repeated-start, descartar bytes de relleno que algunos chips anteponen a sus lecturas (el BMI323, por ejemplo, antepone bytes muertos antes del dato real), o inicializar los pines SDA/SCL, que cada core de Arduino resuelve de forma distinta (ESP32, RP2040 y STM32duino tienen firmas de `begin()` diferentes).

Esa fragmentación tiene un costo directo para el equipo: un driver de sensor escrito para I2C no se reutiliza en SPI sin reescribir su lógica de acceso, un bug de inicialización de bus se corrige en una plataforma y reaparece en otra, y cada integración nueva reinventa el manejo de casos borde (direcciones de 10 bits, buses que quedan bloqueados tras un reset, dispositivos que necesitan re-direccionarse en tiempo de ejecución).

DevLab Interface ataca esto separando dos cosas que solían mezclarse: **el contrato de acceso a registros** (qué significa leer/escribir un registro de N bits) y **el transporte físico** que lo implementa. El driver de un sensor se programa una sola vez contra el contrato; el transporte concreto se elige por bus, no por sensor.

Arquitectura del contrato: `DevLab_BusIO`

El corazón de la biblioteca es una sola clase plantilla, `DevLab_BusIO<Derived>`, resuelta mediante **CRTP** (Curiously Recurring Template Pattern) en tiempo de compilación. La decisión de diseño es deliberada: en lugar de una interfaz virtual (con vtable y despacho en tiempo de ejecución), cada bus concreto se declara como `class DevLab_I2C_Interface : public DevLab_BusIO<DevLab_I2C_Interface>`, y el compilador resuelve las llamadas de forma directa. En microcontroladores con RAM y ciclos limitados, esto elimina el costo de indirección sin renunciar a la abstracción.

Un bus concreto solo tiene que implementar dos primitivas:

- `writeBytes(reg, data, len)`
- `readBytes(reg, data, len)`

Sobre esas dos, `DevLab_BusIO` construye — una sola vez, compartido por todos los buses — el resto del contrato: `readRegister8/16/32`, `writeRegister8/16/32` (little-endian, como la mayoría de los layouts de registros de sensores) y los helpers de bit-field `readBits`, `readBits16`, `readBits32`. Un driver de sensor escrito contra este contrato funciona sin cambios sobre I2C hoy y sobre SPI el día que ese transporte esté disponible.

Otra decisión relevante: **el contrato es header-only, pero los buses concretos no lo son**. `DevLab_I2C_Interface`, `DevLab_I2C10_Interface` y el orquestador son pares `.h/.cpp` normales, no plantillas. Esto mantiene los binarios de firmware pequeños y los tiempos de compilación razonables, algo que importa en proyectos con múltiples sensores y múltiples archivos fuente.

La capa I2C actual

I2C es hoy el único transporte implementado, pero está resuelto en varias capas para no repetir código entre variantes:

`DevLab_I2C_Common` concentra la plomería compartida por todas las variantes de I2C: el puntero a `TwoWire`, el reloj, el conteo de bytes de relleno (“dummy bytes”) a descartar en cada lectura, y la lógica de `begin()`. Es herencia simple, sin virtuales — mismo criterio de costo cero que el contrato —, y evita el copy-paste entre las variantes de 7 y 10 bits.

`DevLab_I2C_Interface` implementa el contrato para direccionamiento I2C estándar de 7 bits: escribe el puntero de registro, hace repeated-start y lee N bytes. Los bytes de relleno que algunos chips anteponen a sus lecturas (el caso de referencia es el Bosch BMI323) se declaran una vez en el constructor y cada helper los descarta automáticamente — el driver del sensor nunca calcula offsets de bytes a mano.

`DevLab_I2C10_Interface` implementa el mismo contrato para direccionamiento de 10 bits (el esquema reservado `0b11110XX` de la especificación I2C, con las dos primeras líneas de dirección en la cabecera y los 8 bits restantes en un segundo byte). Se mantiene como clase separada — en vez de un flag en tiempo de ejecución dentro de `DevLab_I2C_Interface` — para que las transacciones de 7 bits, que son la inmensa mayoría, nunca paguen el costo de una verificación que no necesitan.

Recuperación de bus, agregada recientemente: si un dispositivo queda a mitad de una transacción (por ejemplo, tras un reset), SDA puede quedar retenido en bajo y bloquear el bus para todos los demás dispositivos. `DevLab_I2C_Common` implementa la secuencia de recuperación estándar (hasta 18 pulsos de reloj manuales en SCL para liberar al esclavo, seguidos de una condición STOP), expuesta como `devlabBeginI2CBusRecovered()`. El orquestador ya la usa a través de `beginRecovered()`; es candidata natural para exponerse también en `DevLab_I2C_Interface/DevLab_I2C10_Interface` si el equipo lo necesita en dispositivos de dirección fija.

El orquestador I2C y el protocolo DevLab DDP

`DevLab_I2C_Interface` y `DevLab_I2C10_Interface` asumen un dispositivo de dirección fija — el modelo típico de un sensor. Pero hay una clase de dispositivos DevLab que necesita otra cosa: escanear el bus, identificar qué hay conectado, y re-direccionar dispositivos en tiempo de ejecución. Para eso existe `DevLab_I2C_Orchestrator`, pensado explícitamente para protocolos

a nivel de bus como **DevLab DDP**.

Su modelo de transacción es distinto al de una interfaz de registros: en vez de “escribir registro + repeated-start + leer”, sigue un ciclo de **comando/respuesta** — escribe un byte de comando (con STOP), deja que el dispositivo tome su tiempo de procesamiento, y luego emite una lectura nueva. El orquestador no asume una respuesta inmediata; el dispositivo decide cuándo está listo. Expone tres operaciones: `ping()` (verificar si una dirección responde), `writeCommand()` (comando sin retorno) y `transact()` (comando con retorno, con retardo configurable).

A diferencia de las interfaces de registro, una sola instancia del orquestador habla con **cualquier dirección del bus** — no guarda una dirección propia —, lo que lo hace la pieza natural para escanear el bus buscando dispositivos DevLab, identificarlos y asignarles dirección en frío. Esto es lo que habilita, a nivel de infraestructura, un modelo de **dispositivos plug-and-play**: un módulo DevLab puede conectarse al bus sin dirección fija preasignada de fábrica y ser descubierto por el host en tiempo de ejecución.

El orquestador es también el primer consumidor de la recuperación de bus (`beginRecovered()`), coherente con su rol: si va a escanear un bus compartido por varios dispositivos, es el punto donde un esclavo atascado tiene más probabilidad de aparecer.

Compatibilidad de plataformas

La inicialización de pines SDA/SCL es uno de los puntos donde más diverge cada core de Arduino, y es exactamente el tipo de detalle que la biblioteca absorbe en `DevLab_I2C_Common::beginCommon()` para que el resto del código no tenga que pensarlo:

Plataforma	Estado	Notas
ESP32 (incl. C6 / C5 / H2)	Probado	<code>Wire.begin(sda, scl, clock)</code> en una sola llamada
RP2040 (arduino-pico)	Probado	Pines se fijan antes de <code>begin()</code> , sin argumentos
STM32 (STM32duino core)	Probado	<code>setSDA()/setSCL()</code> antes de <code>begin()</code> ; incorporado en la versión 1.1.2
Otras placas Arduino-compatibles con AVR	Probado	Usa <code>Wire.begin()</code> estándar sin pines explícitos

En la práctica, el trabajo de mantenimiento recurrente de esta capa: cada nuevo core de Arduino que el equipo quiera soportar entra aquí, una sola vez, y queda disponible para todos los buses y todos los drivers que se construyan sobre el contrato.

En desarrollo y hoja de ruta

SPI concreto (`DevLab_SPI_Interface`) — **planeado, es el siguiente hito**. El contrato ya está diseñado para ser agnóstico de transporte, falta la clase concreta que lo formalice dentro de este repositorio, en paralelo a como hoy existen `DevLab_I2C_Interface` y `DevLab_I2C10_Interface`.

Recuperación de bus fuera del orquestador. `devlabBeginI2CBusRecovered()` ya vive en `DevLab_I2C_Common` y hoy solo se expone públicamente a través de `DevLab_I2C_Orchestrator::beginRecovered()`. Extender ese mismo `beginRecovered()` a `DevLab_I2C_Interface` y `DevLab_I2C10_Interface` es un cambio pequeño y ya soportado por

la base común — relevante para cualquier sensor de dirección fija que pueda quedar en un bus compartido con dispositivos DDP.

Para qué servirá: casos de uso concretos

Portar un driver de sensor entre buses sin reescribirlo. Un driver contra `DevLab_BusIO` (como `DevLab_BMI323`, ya usado en los ejemplos de referencia) se valida hoy sobre I2C y se reutiliza sobre SPI en cuanto exista `DevLab_SPI_Interface`, sin tocar la lógica del sensor. Para el equipo, esto significa que la elección de bus en una placa nueva deja de ser una decisión que obliga a reescribir drivers.

Dispositivos DevLab plug-and-play (DDP). El orquestador es la base para que módulos DevLab se descubran, identifiquen y direccionen en el bus sin configuración manual de direcciones — el tipo de infraestructura que necesita cualquier producto que quiera soportar “conecta y listo” entre múltiples módulos en el mismo bus I2C.

Integración rápida de sensores de terceros. El manejo de bytes de relleno por dispositivo (declarado una vez en el constructor) y el `getWhoAmI()` de validación genérica bajan el costo de traer un sensor externo nuevo al ecosistema: no hay que escribir código de bus desde cero, solo mapear sus registros sobre el contrato existente.

Firmware resiliente en campo. La recuperación de bus I2C importa en cualquier producto donde el dispositivo pueda reiniciarse o desconectarse sin que el host lo controle (un módulo que se hot-plug, un sensor que se resetea por ruido eléctrico): sin ella, un bus atascado requiere ciclar la alimentación completa del sistema para recuperarse.

Portabilidad entre líneas de producto con distintos MCU. Con ESP32, RP2040, STM32 y AVR ya cubiertos, un mismo driver de sensor corre sin cambios en productos que usan distintas familias de microcontrolador.